

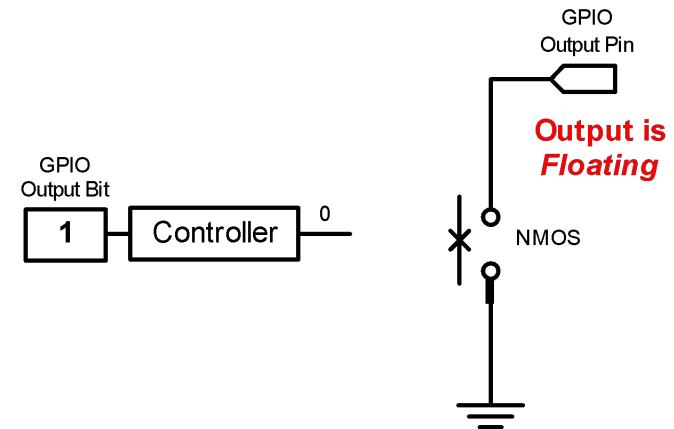
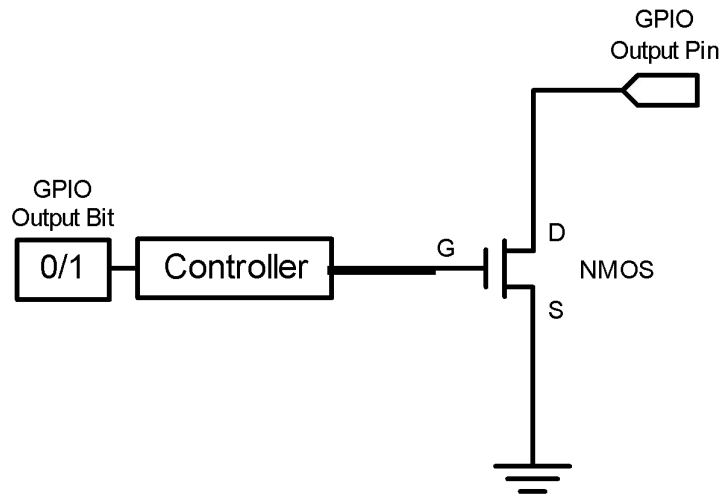
Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C

CSE33 I : Microprocessor Interfacing and Embedded Systems

Lecture#10: GPIO Part#2 (Chapter#14)
Summer, 2023

Mohammad A. Qayum, Ph.D.
ECE, NSU, Dhaka, Bangladesh

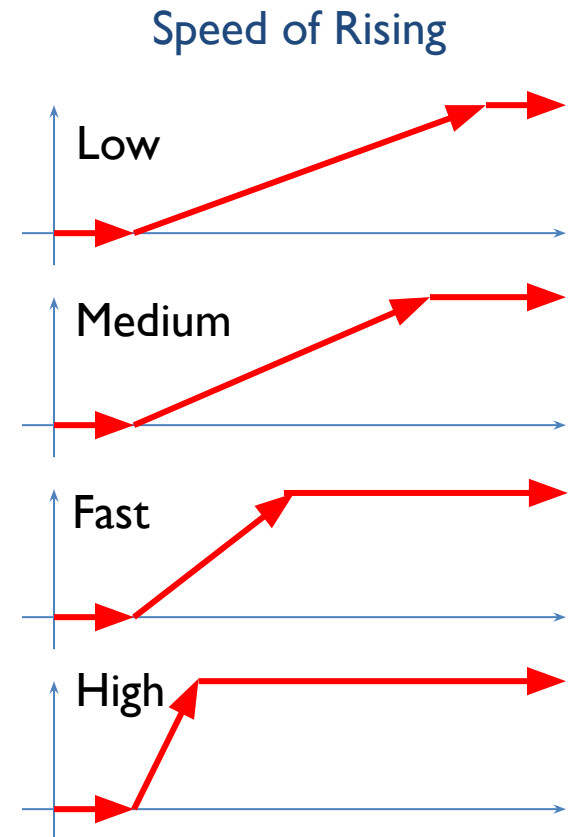
GPIO Output: Open-Drain



Output = 1
GPIO Pin has high-impedance to external circuit

GPIO Output Speed

- Output Speed:
 - Speed of rising and falling
 - Four speeds: Low, Medium, Fast, High
- Tradeoff
 - Higher GPIO speed increases EMI noise and power consumption
 - Configure based on peripheral speed
 - Low speed for toggling LEDs
 - High speed for SPI

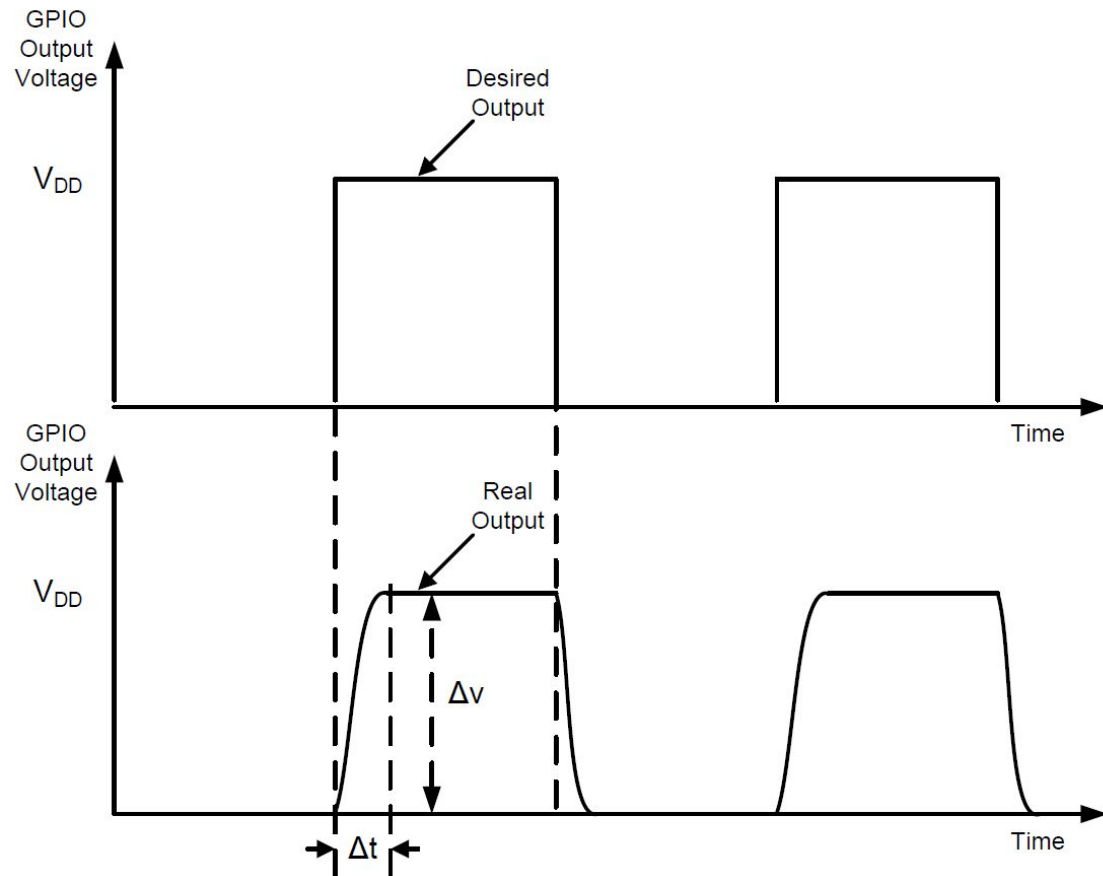


Slew Rate

Slew Rate:
Maximum rate of change of the
output voltage

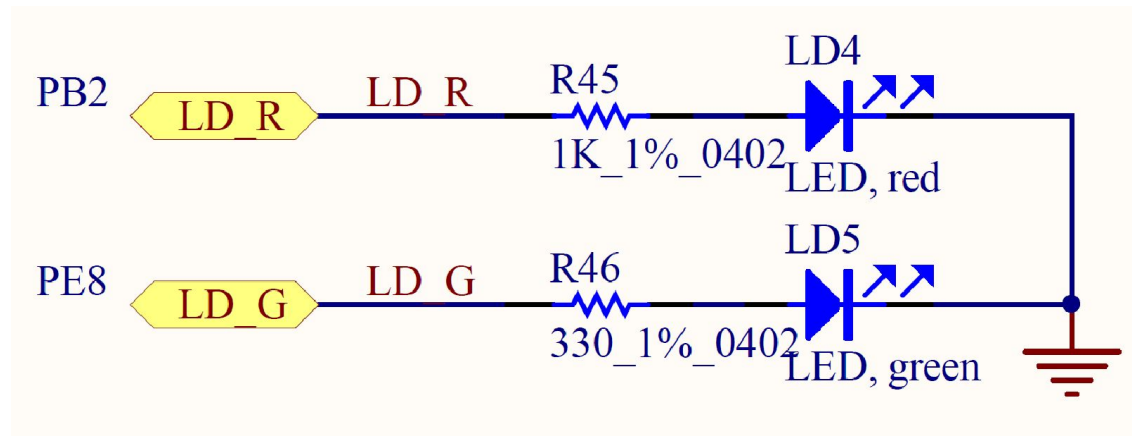
$$\text{Slew Rate} = \max \left(\frac{\Delta V}{\Delta t} \right)$$

A high slew rate allows the
output to be toggled at a fast
speed.



GPIO Output: Push-Pull vs Open-Drive

Output Bit	Push-Pull	Open-Drain
1	High	HiZ
0	Low	Low



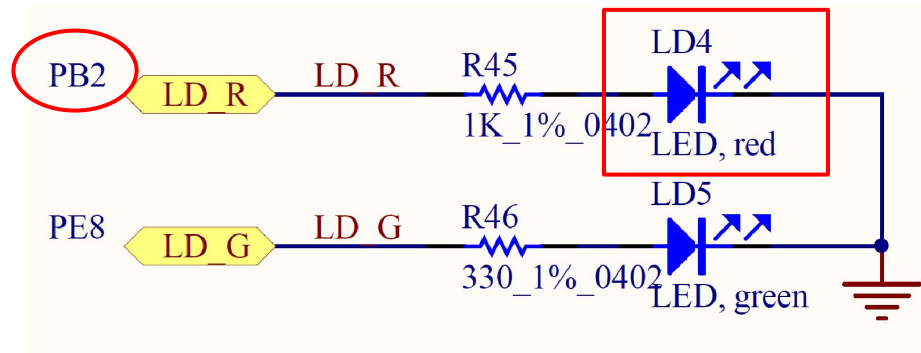
Use push-pull output, instead of open-drain output!

GPIO Output Data Register (ODR)

- 16 bits reserved, 16 data bits, 1 bit for each pin

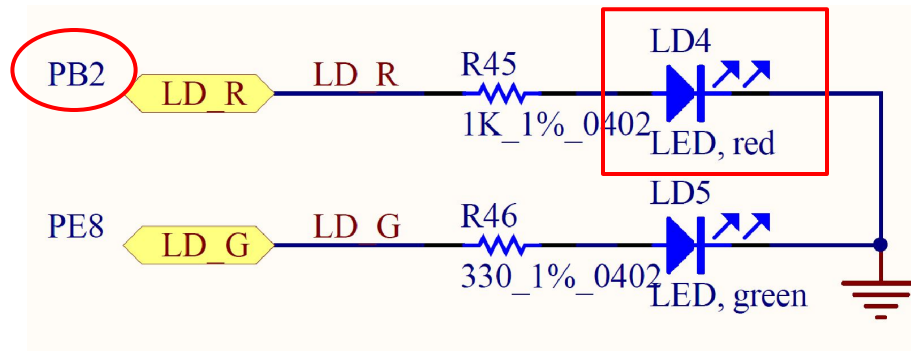
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 2



```
GPIOB->ODR |= 1UL << 2; // Set bit 2
```

Light up the Red LED (PB.2)



```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Enable clock of Port B

GPIOB->MODER &= ~(3UL<<4); // Clear mode bits
GPIOB->MODER |= 1UL<<4; // Set mode to output

GPIOB->OTYPE &= ~(1UL<<2); // Select push-pull output

GPIOB->ODR |= 1UL << 2; // Output 1 to turn on red LED
```

GPIO Initialization

- Turn on the clock to the GPIO Port (e.g. Port B)

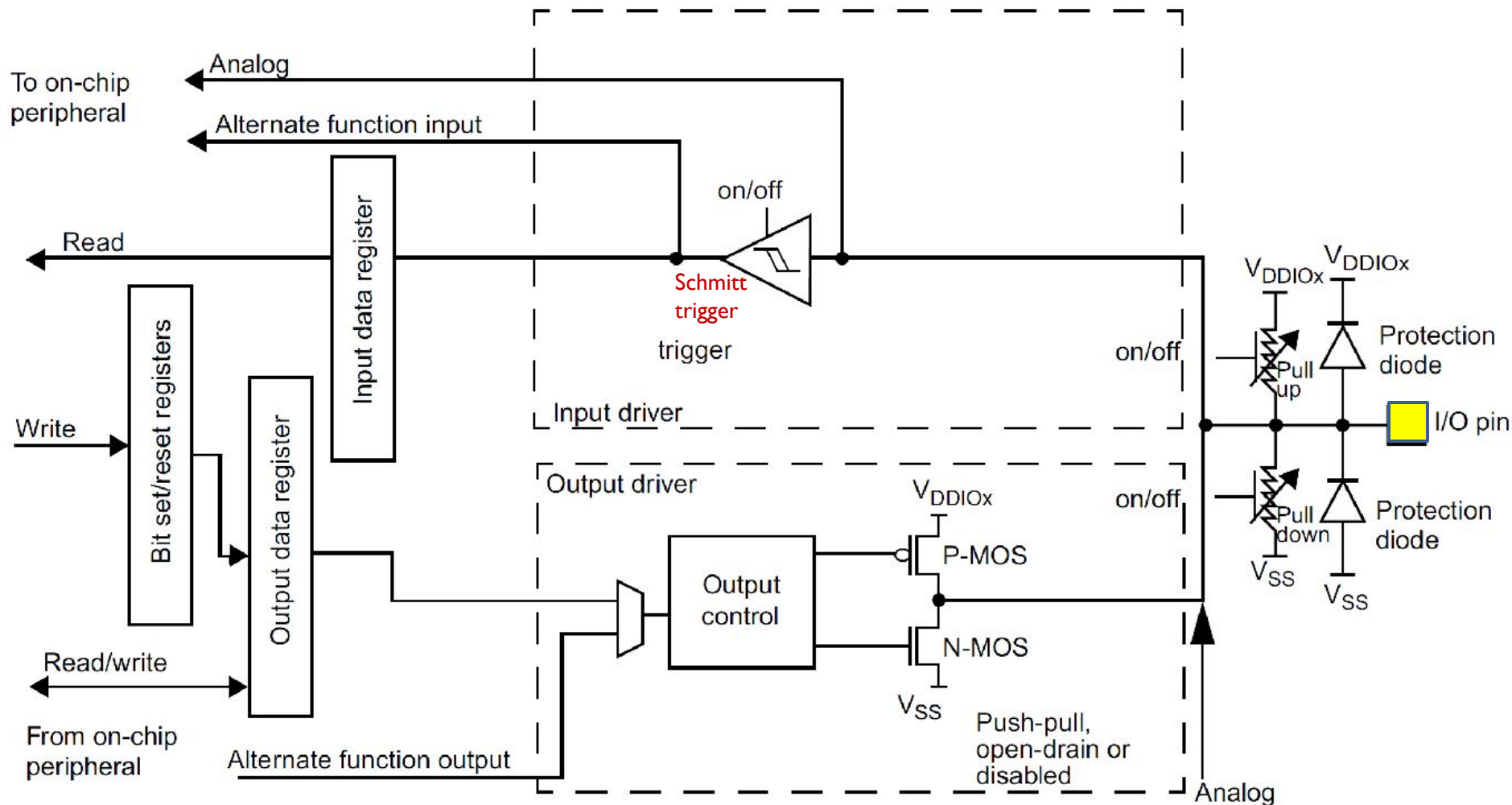
RCC->AHBENR |= RCC_AHBENR_GPIOBEN; Reset and Clock Control (RCC)

- Configure GPIO mode, output type, speed, pull-up/pull-down

```
typedef struct
{
    __IO uint32_t MODER;
    __IO uint16_t OTYPER;
    uint16_t RESERVED0;
    __IO uint32_t OSPEEDR;
    __IO uint32_t PUPDR;
    __IO uint16_t IDR;
    uint16_t RESERVED1;
    __IO uint16_t ODR;
    uint16_t RESERVED2;
    __IO uint16_t BSRRL; /* BSRR register is split to 2 * 16-bit fields BSRRL */
    __IO uint16_t BSRRH; /* BSRR register is split to 2 * 16-bit fields BSRRH */
    __IO uint32_t LCKR;
    __IO uint32_t AFR[2];
} GPIO_TypeDef;
#define PERIPH_BASE ((uint32_t)0x40000000)
#define AHBPERIPH_BASE (PERIPH_BASE + 0x20000)
#define GPIOB_BASE (AHBPERIPH_BASE + 0x0400)
#define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
```

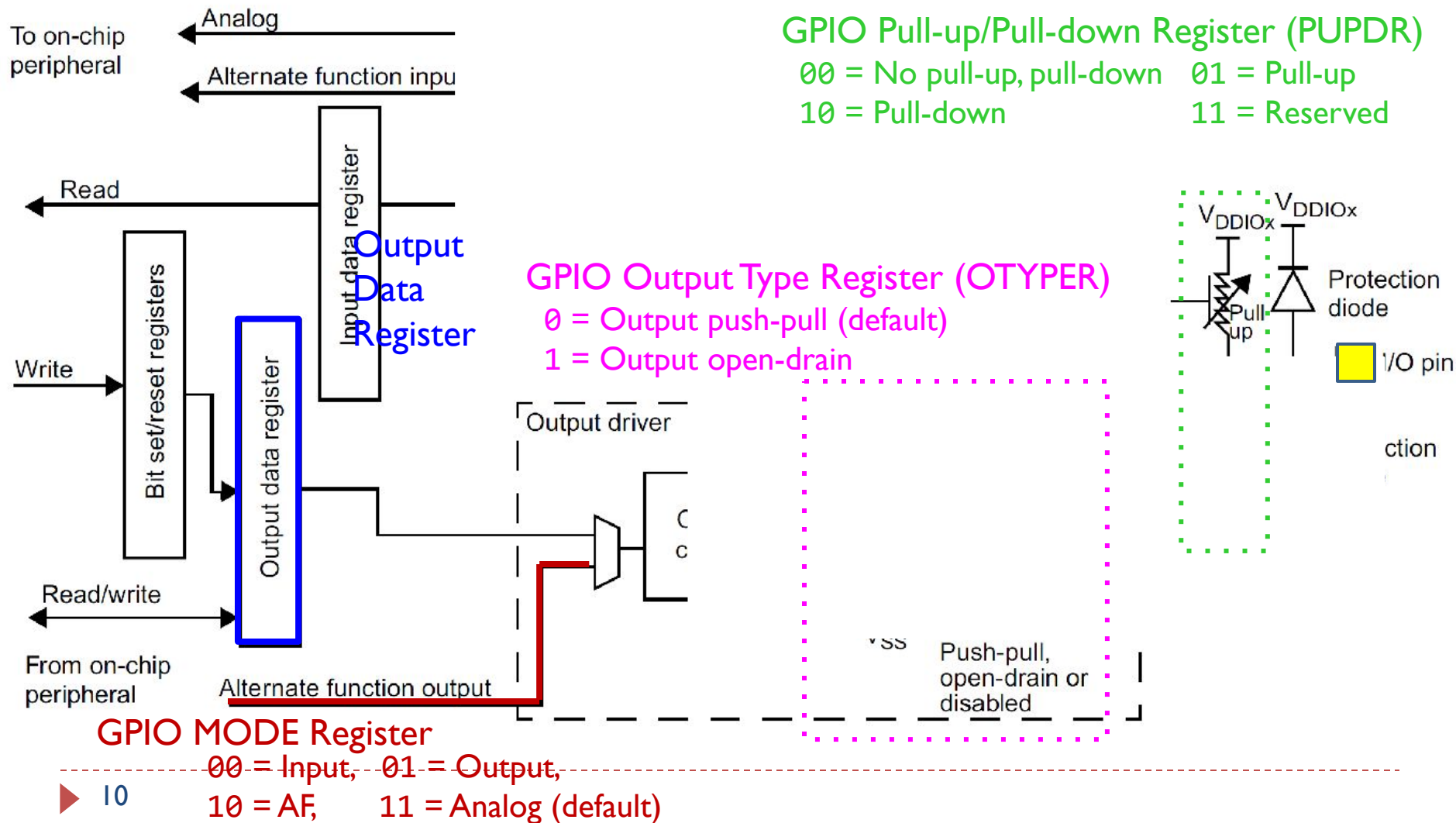

Basic Structure of an I/O Port Bit

Input and Output



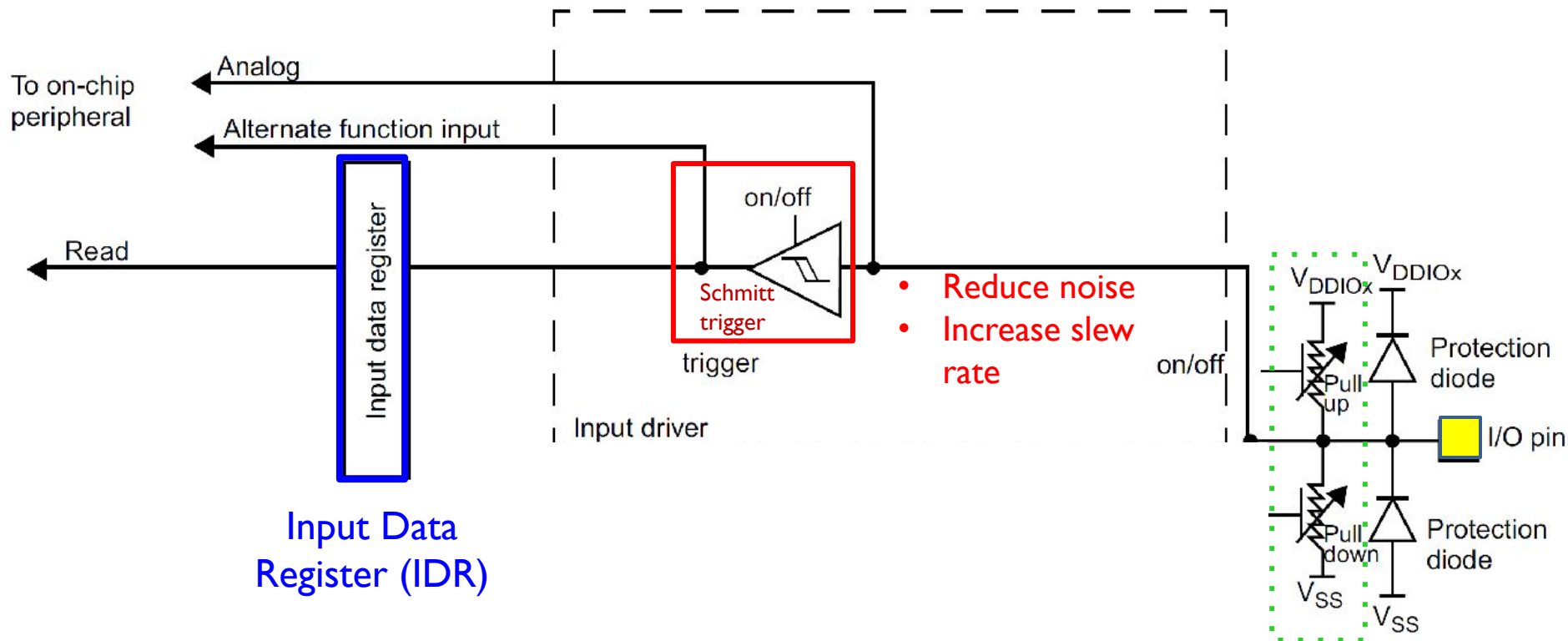
Basic Structure of an I/O Port Bit:

Output



Basic Structure of an I/O Port Bit:

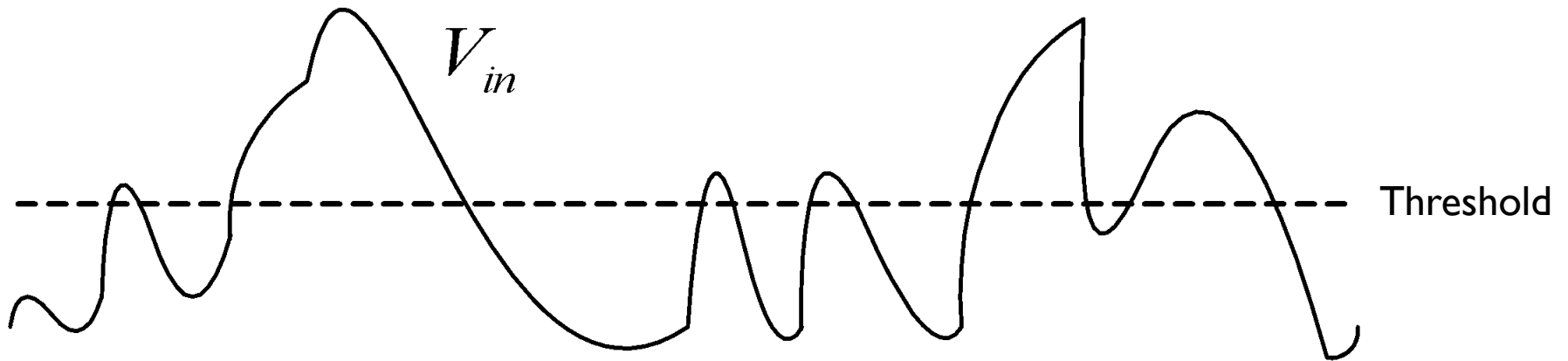
Input



GPIO Pull-up/Pull-down Register (PUPDR)

00 = No pull-up, pull-down 01 = Pull-up
10 = Pull-down 11 = Reserved

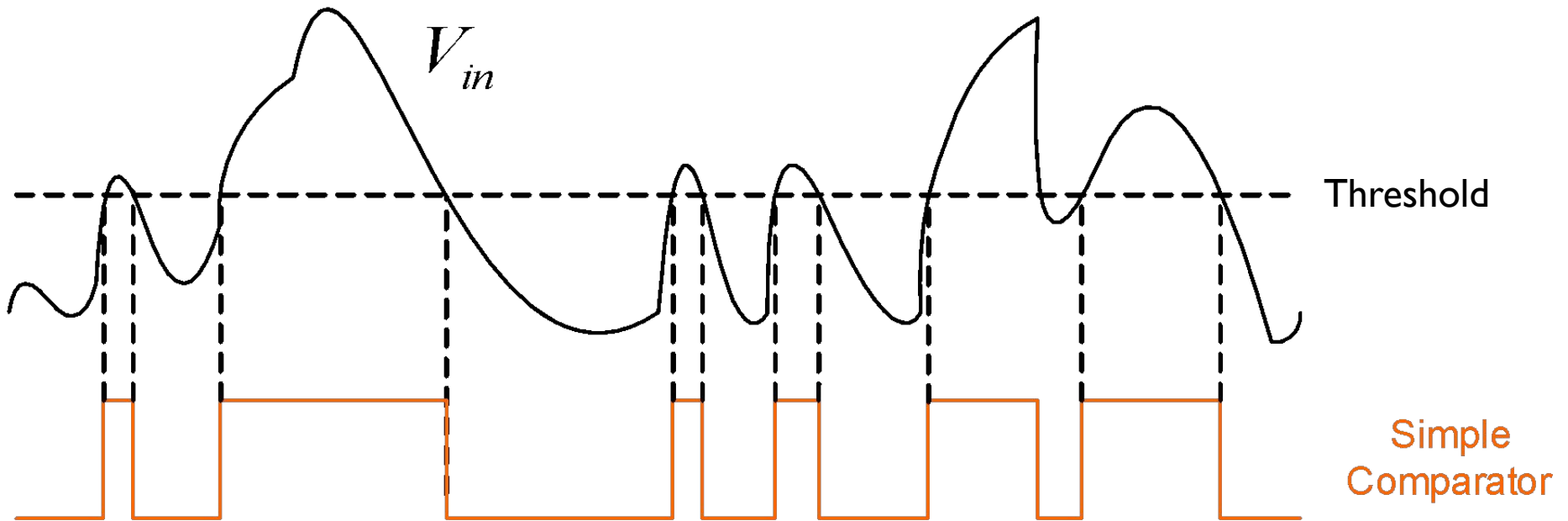
Schmitt Trigger



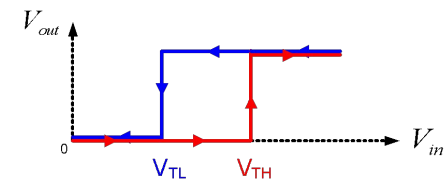
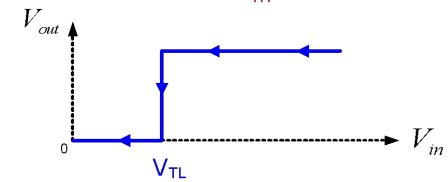
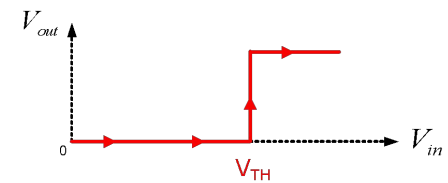
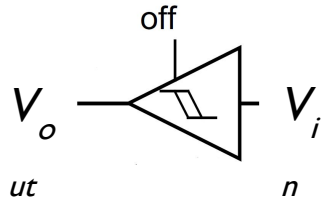
Analog signals

- Noisy
- Rise and fall slowly (small slew rate)

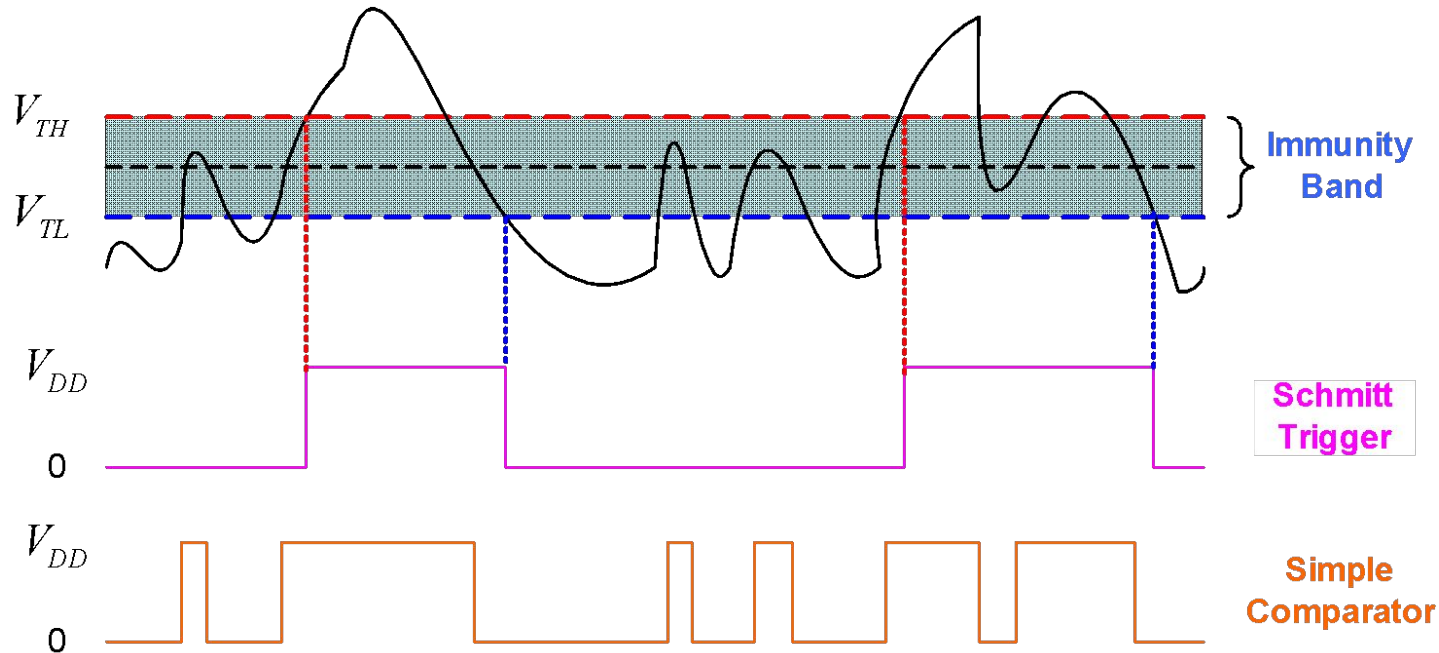
Schmitt Trigger



Schmitt Trigger



Trigger Low Trigger High



Enable Clock

□ AHB2 peripheral clock enable register (RCC_AHB2ENR)

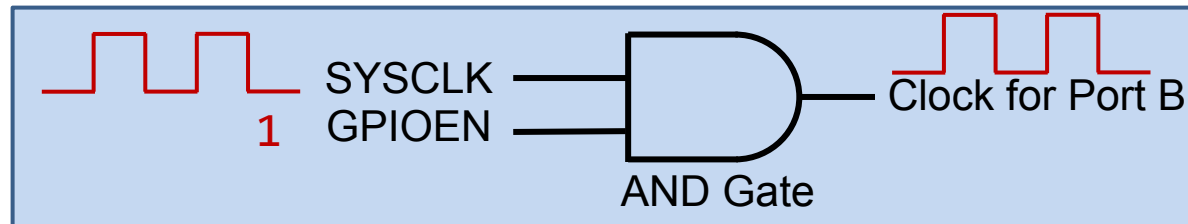
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	Res.	AESEN
													rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADCEN	OTGFS EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw

Bit 1 **GPIOBEN**: IO port B clock enable

Set and cleared by software.

0: IO port B clock disabled

1: IO port B clock enabled



```
#define RCC_AHB2ENR_GPIOBEN ((uint32_t)0x0000002U)
```

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
```

GPIO Pull-up/Pull-down Register (PUPDR)

16 pins per port, 2 bits per pin

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits $2y+1:2y$ **PUPDy[1:0]**: Port x configuration bits ($y = 0..15$)

Pin 2

Pin 1

Pin 0

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

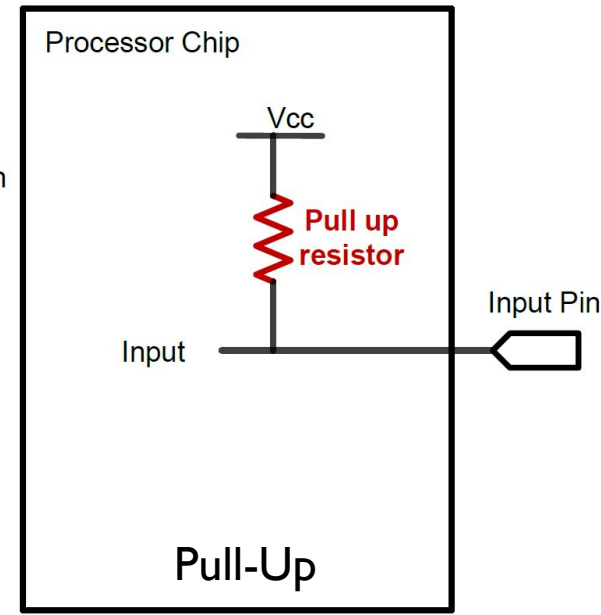
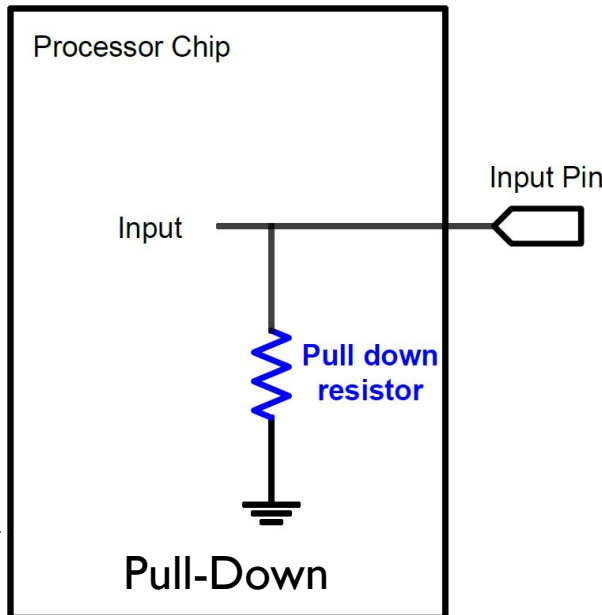
01: Pull-up

10: Pull-down

11: Reserved

// No pull-up, pull-down

GPIOA->PUPDR &= ~3UL;



GPIO Input Data Register (IDR)

- 16 bits reserved, 16 data bits (1 bit per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

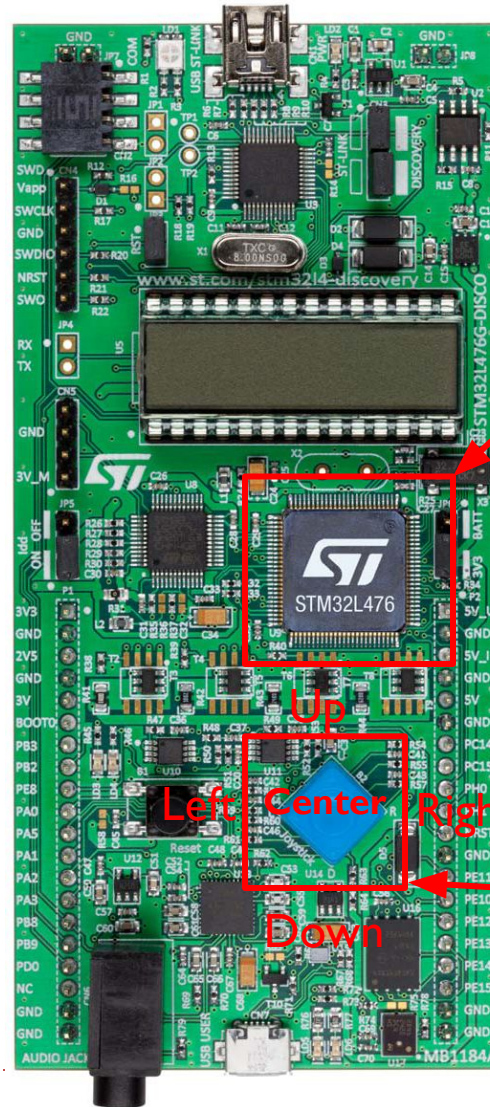
```
// Demo of reading pin 7
uint32_t mask = 1UL<<7;
uint32_t input = (GPIOA->IDR & mask) == mask;
```

or

```
uint32_t input = (GPIOA->IDR & mask) >> 7;
```

Joystick

STM32L4 Discovery Kit

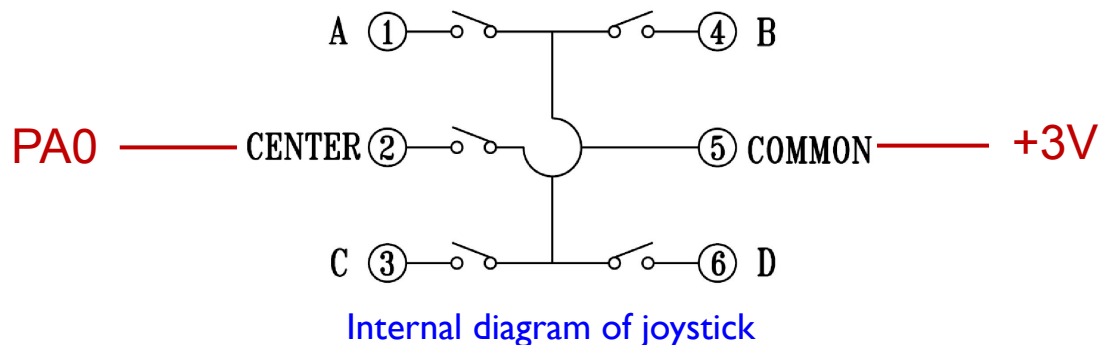
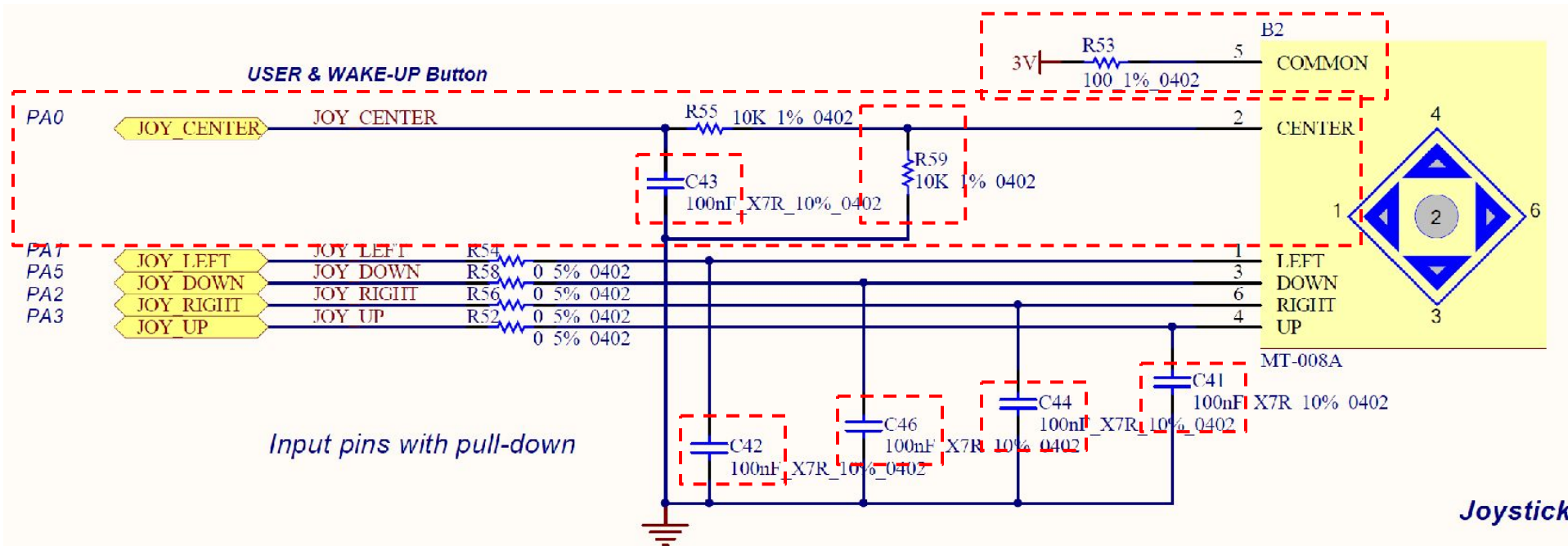


STM32L4

Up
Left Center Right
Down

Joystick with
4-direction
control and
selector

Joystick



Read Input of Pin PA.0

```
uint32_t input;

RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN; // Enable clock of Port A

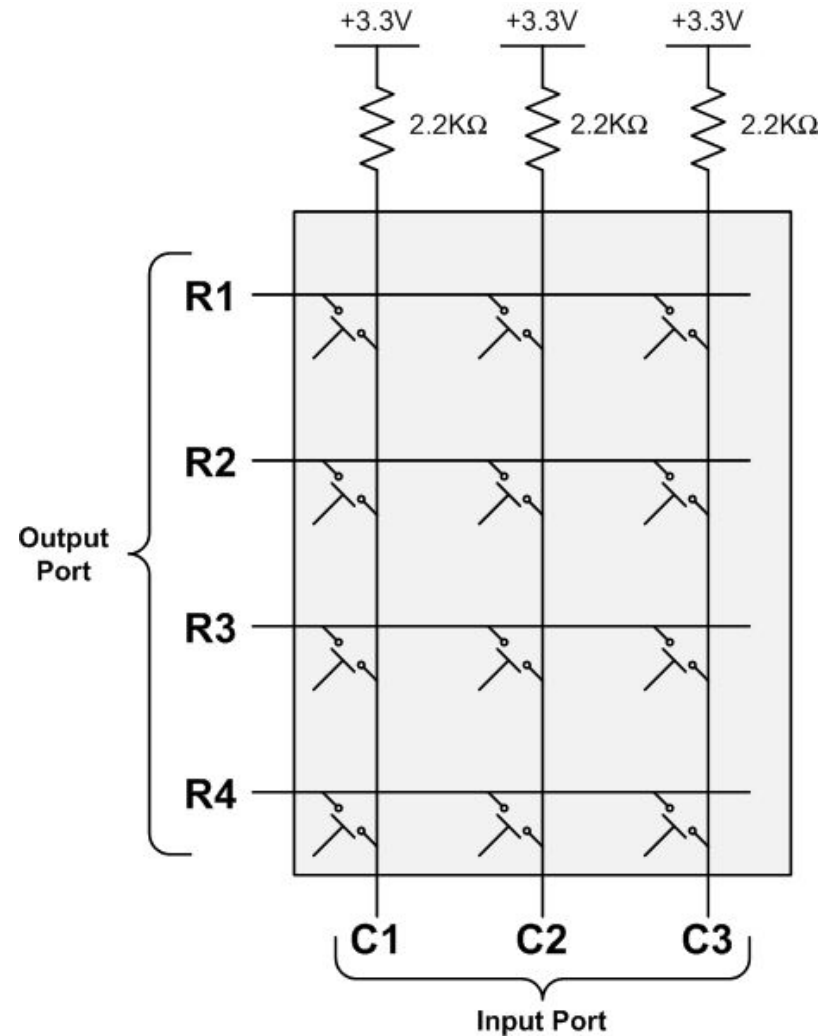
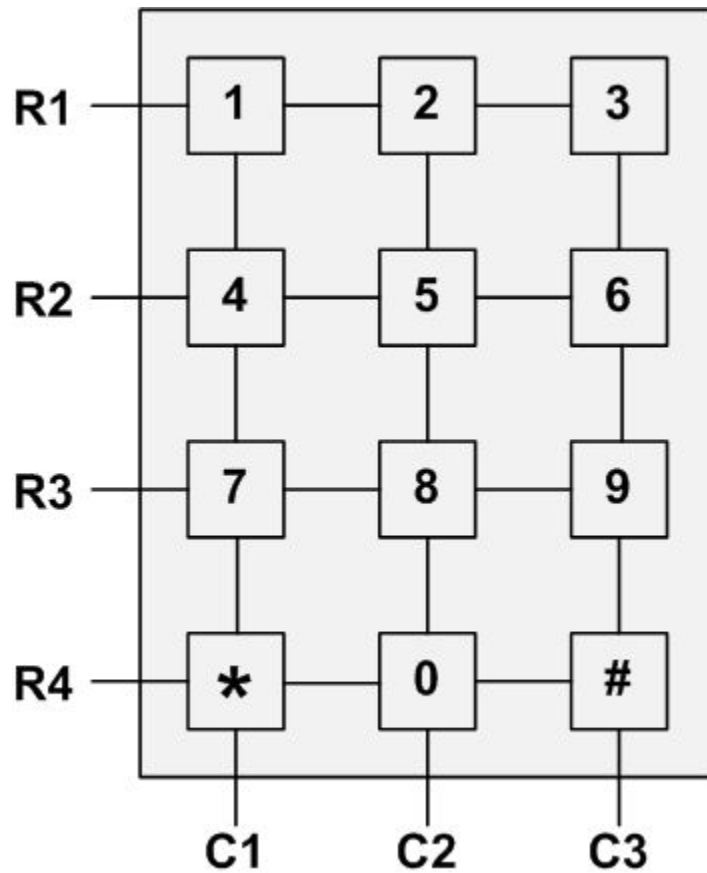
GPIOA->MODER &= ~3UL; // Set PA.0 as digital input

GPIOA->PUPDR &= ~3UL; // No pull-up, no pull-down

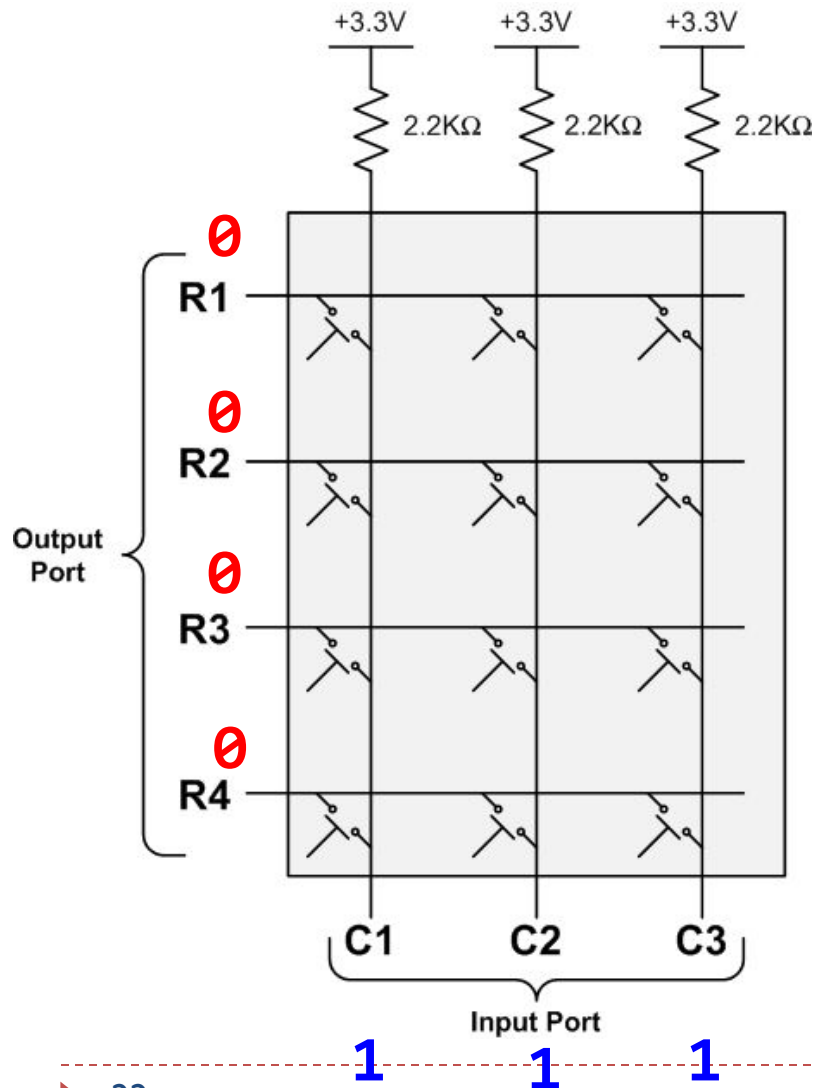
// Read pin 0
input = (GPIOA->IDR & 1UL);

if (input == 0) {
    // Center of joystick is not pressed
    ...
} else {
    // Center of joystick is pressed
    ...
}
```

Keypad Scan



Keypad Scan

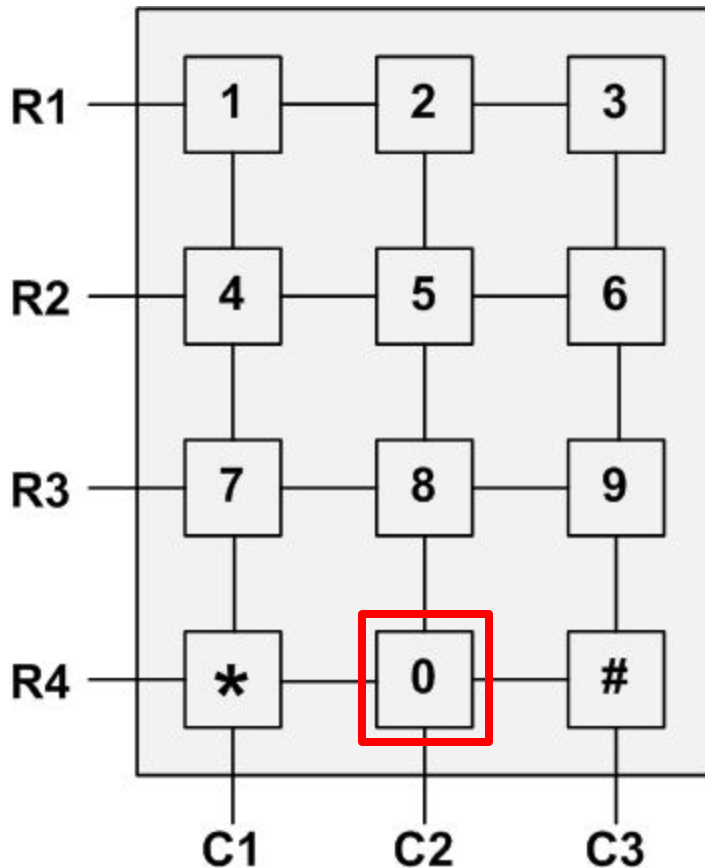


Step 1: Set Output
 $R1, R2, R3, R4 = 0000$

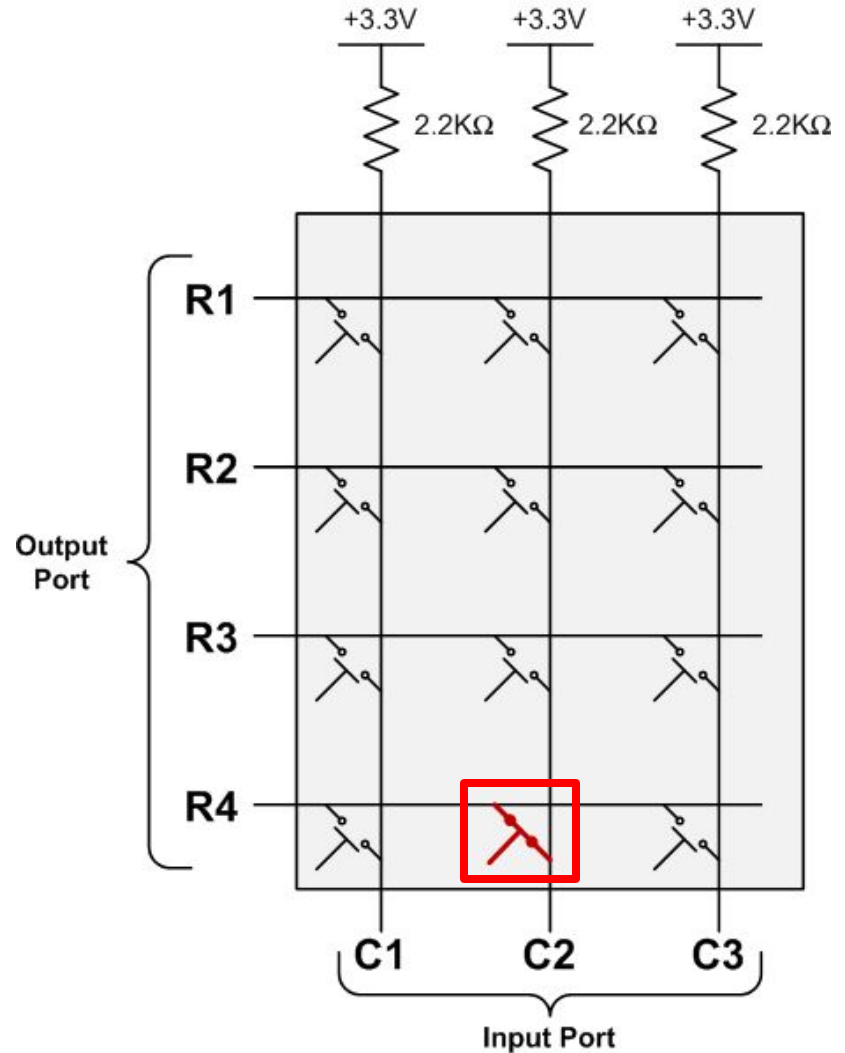
Step 2: Read Input
 $C1, C2, C3 = 111$

⇒ No key pressed

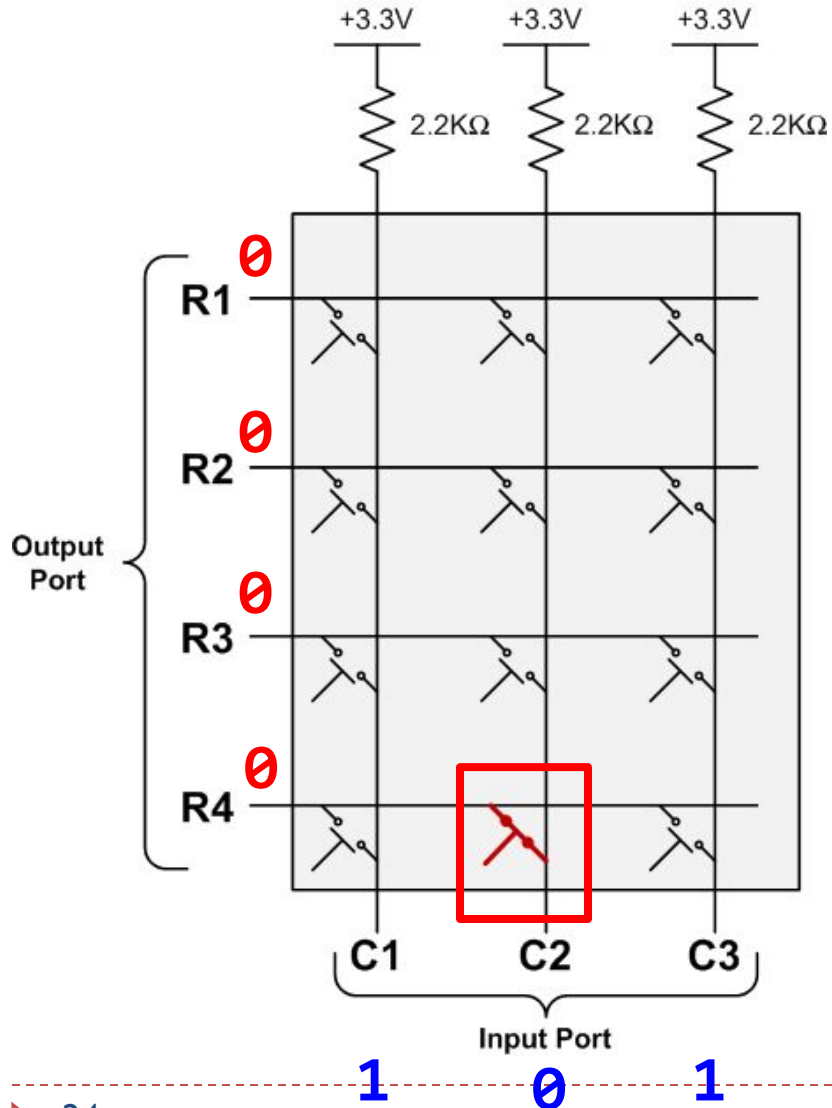
Keypad Scan



Key "0" is pressed



Keypad Scan



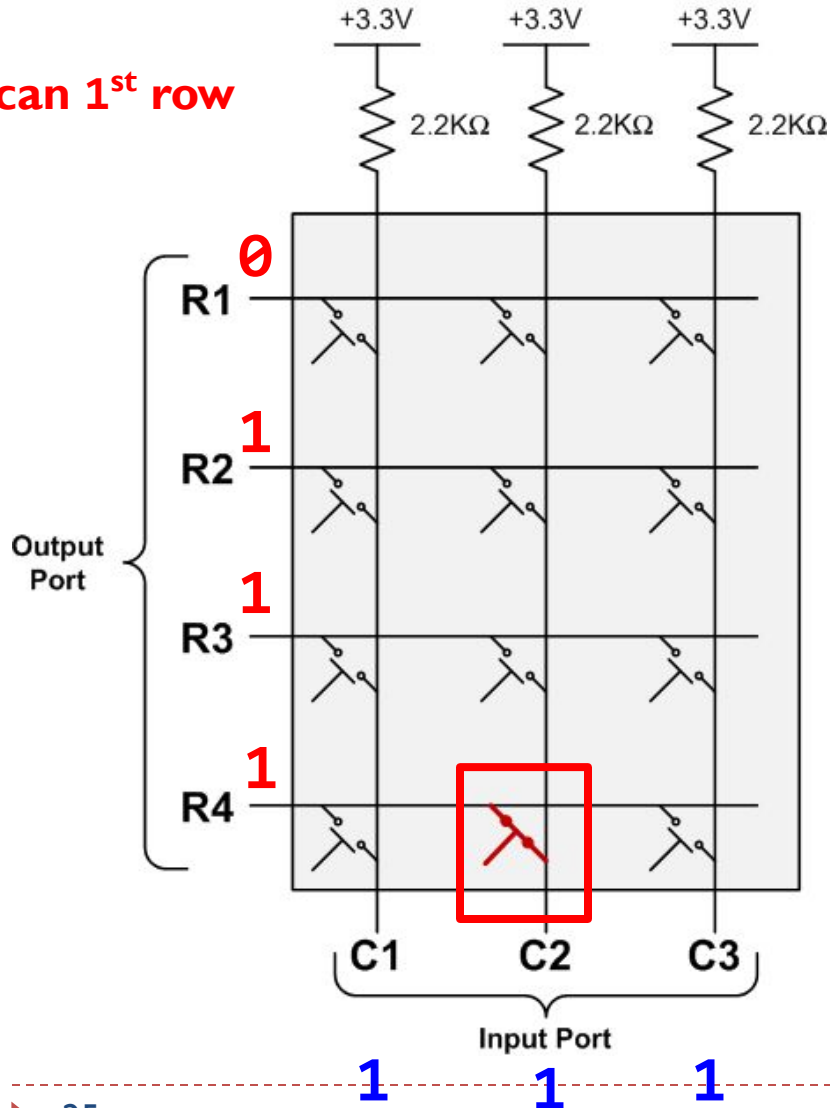
Step 1: Set Output
 $R1, R2, R3, R4 = 0000$

Step 2: Read Input
 $C1, C2, C3 = 101$

⇒ Some key in 2nd column is pressed down

Keypad Scan

Scan 1st row



Step 1: Set Output

$R1, R2, R3, R4 = 0000$

Step 2: Read Input

$C1, C2, C3 = 101$

→ Step 3a: Scan 1st row

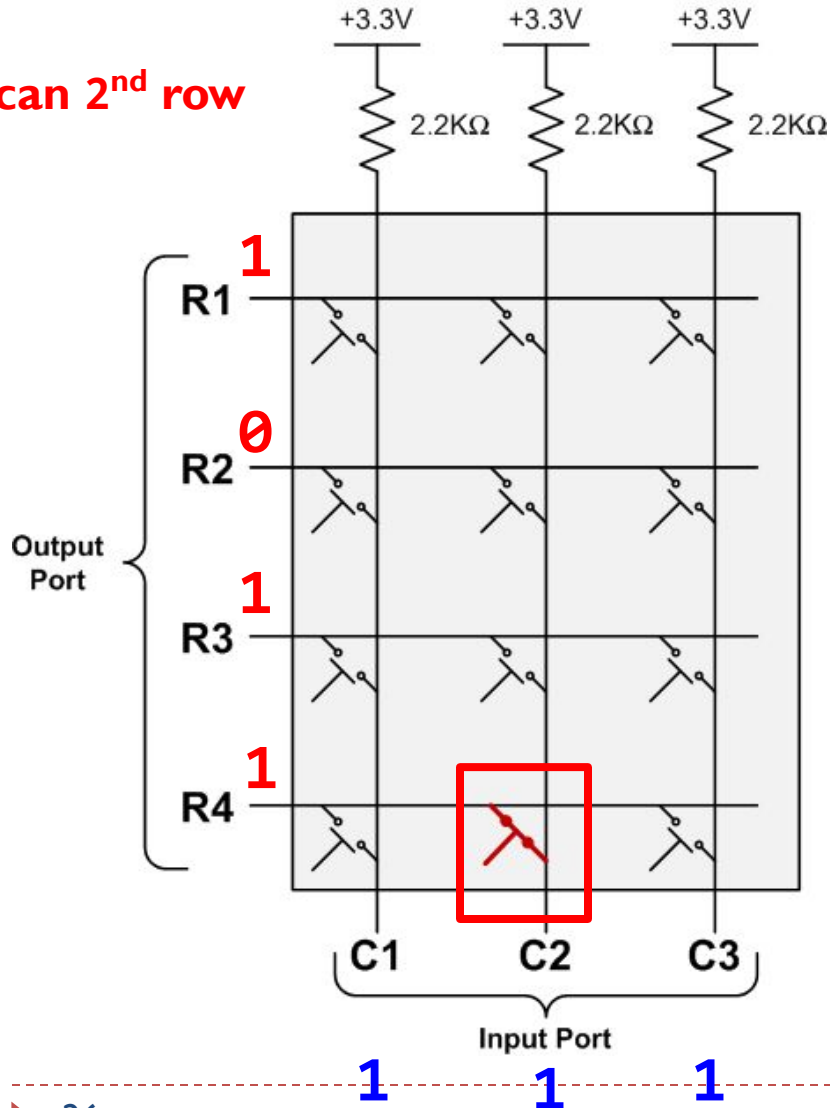
$R1, R2, R3, R4 = 0111$

$C1, C2, C3 = 111$

⇒ No key in 1st row
is pressed down

Keypad Scan

Scan 2nd row



Step 1: Set Output
 $R1, R2, R3, R4 = 0000$

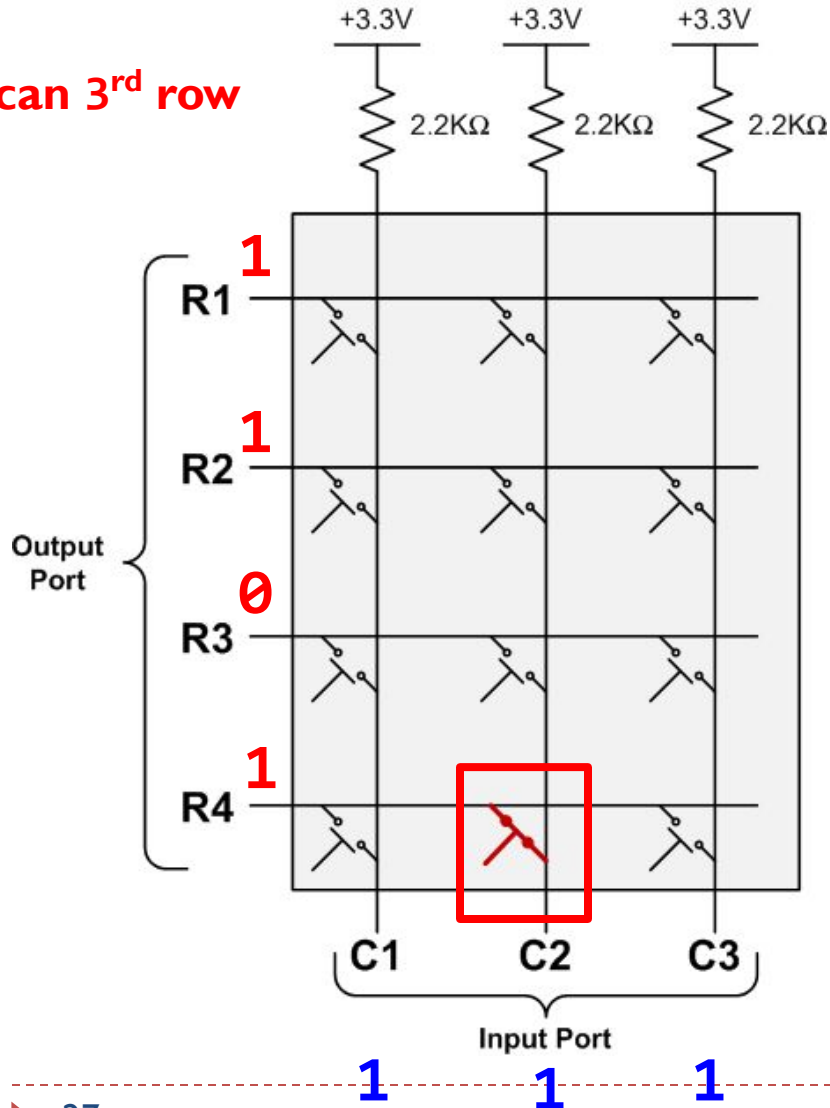
Step 2: Read Input
 $C1, C2, C3 = 101$

→ Step 3b: Scan 2nd row
 $R1, R2, R3, R4 = 1011$
 $C1, C2, C3 = 111$

⇒ No key in 2nd row
is pressed down

Keypad Scan

Scan 3rd row



Step 1: Set Output

$R1, R2, R3, R4 = 0000$

Step 2: Read Input

$C1, C2, C3 = 101$

→ Step 3c: Scan 3rd row

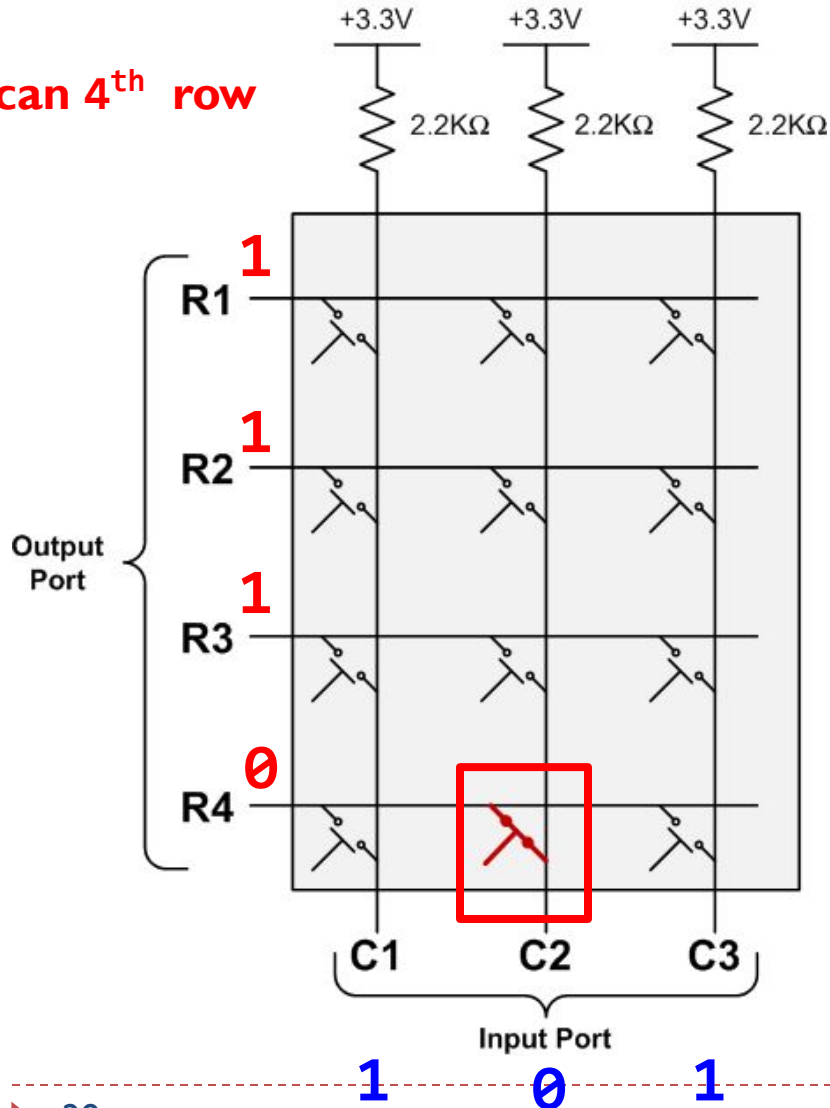
$R1, R2, R3, R4 = 1101$

$C1, C2, C3 = 111$

⇒ No key in 3rd row
is pressed down

Keypad Scan

Scan 4th row



Step 1: Set Output

$R1, R2, R3, R4 = 0000$

Step 2: Read Input

$C1, C2, C3 = 101$

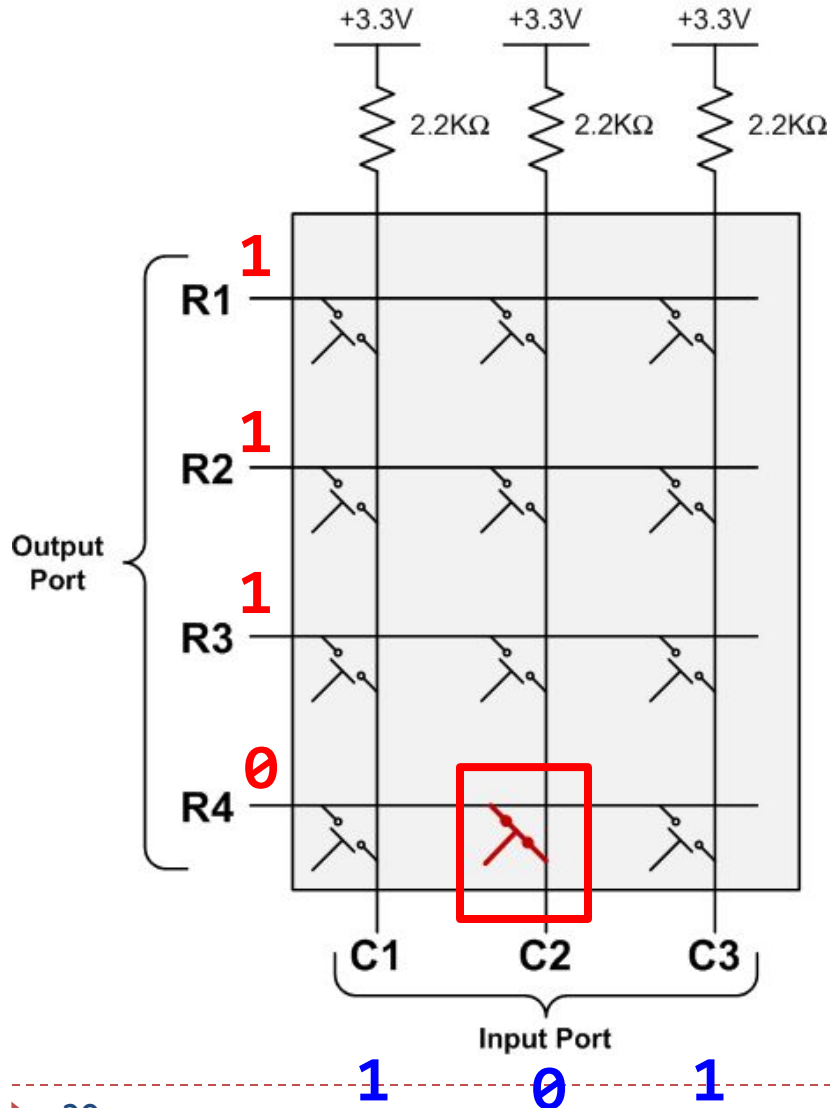
→ Step 3d: Scan 4th row

$R1, R2, R3, R4 = 1110$

$C1, C2, C3 = 101$

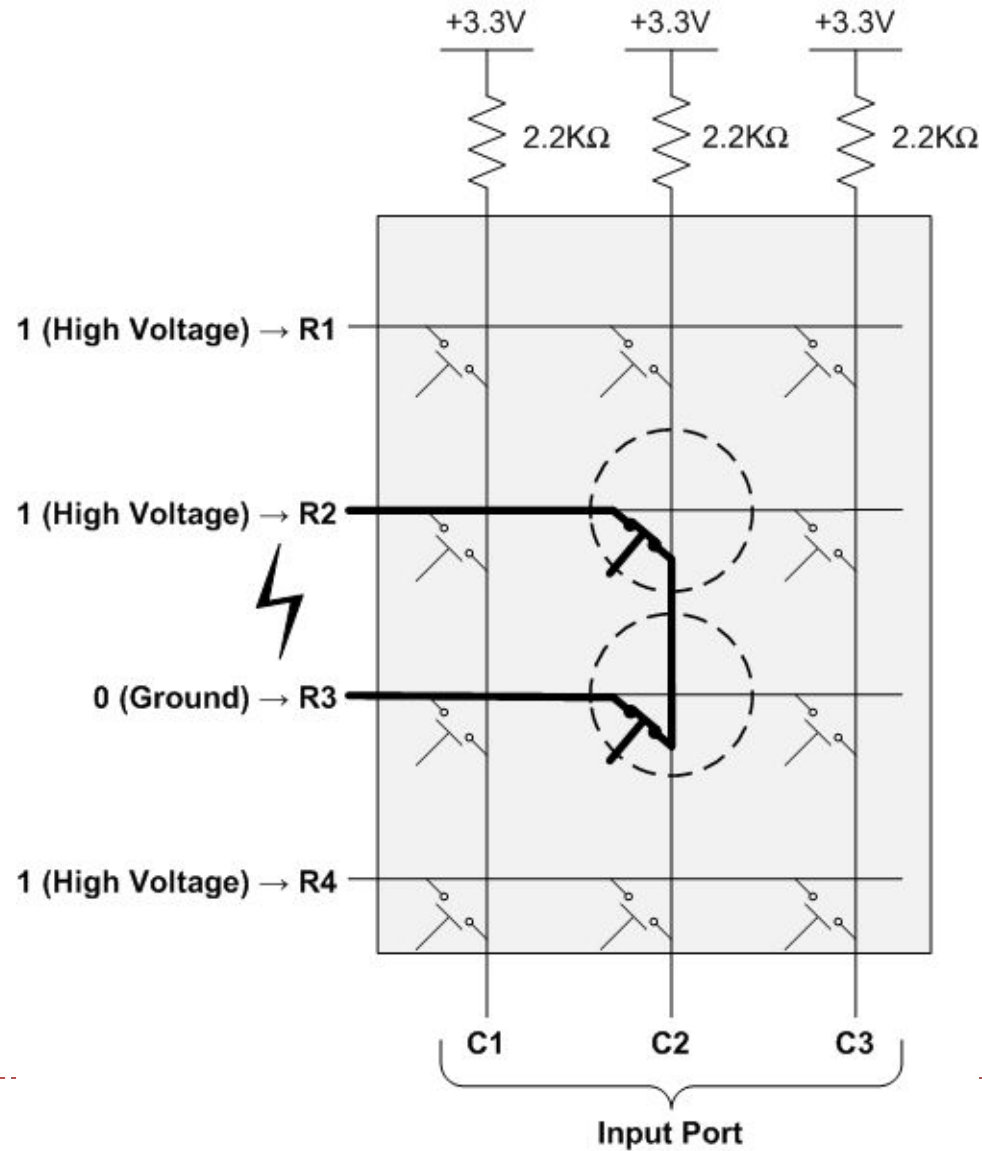
⇒ key in 4th row
is pressed down

Keypad Scan



⇒ Key pressed is located at the second column and the fourth row.

Keypad Scan



I/O Debouncing

- Example signal when a button is pressed

